IT357 Lab Assignment 1

Nithin S
221IT085

Q1. Implement a program that takes an image, applies Canny edge detection, and allows you to adjust the threshold values using track bars.

```
import os
import tkinter as tk
from tkinter import filedialog, messagebox
import cv2
import numpy as np
from PIL import Image, ImageTk

class MinimalCannyDetector:
    def __init__(self, root):
        self.root = root
        self.root.title("Canny Edge Detection")
        self.root.geometry("900x700")
        self.root.configure(bg='white')

        self.original_image = None
        self.processed_image = None
        self.filter_applied = False
        self.low_threshold = tk.IntVar(value=50)
        self.high_threshold = tk.IntVar(value=150)

        self.setup_ui()

    def setup_ui(self):
        main_frame = tk.Frame(self.root, bg='white')
        main_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        self.control_frame = tk.Frame(main_frame, bg='white')
        self.control_frame.pack(pady=(0, 20))

        self.load_btn = tk.Button(self.control_frame, text="Load Image",
                                  command=self.load_image,
```

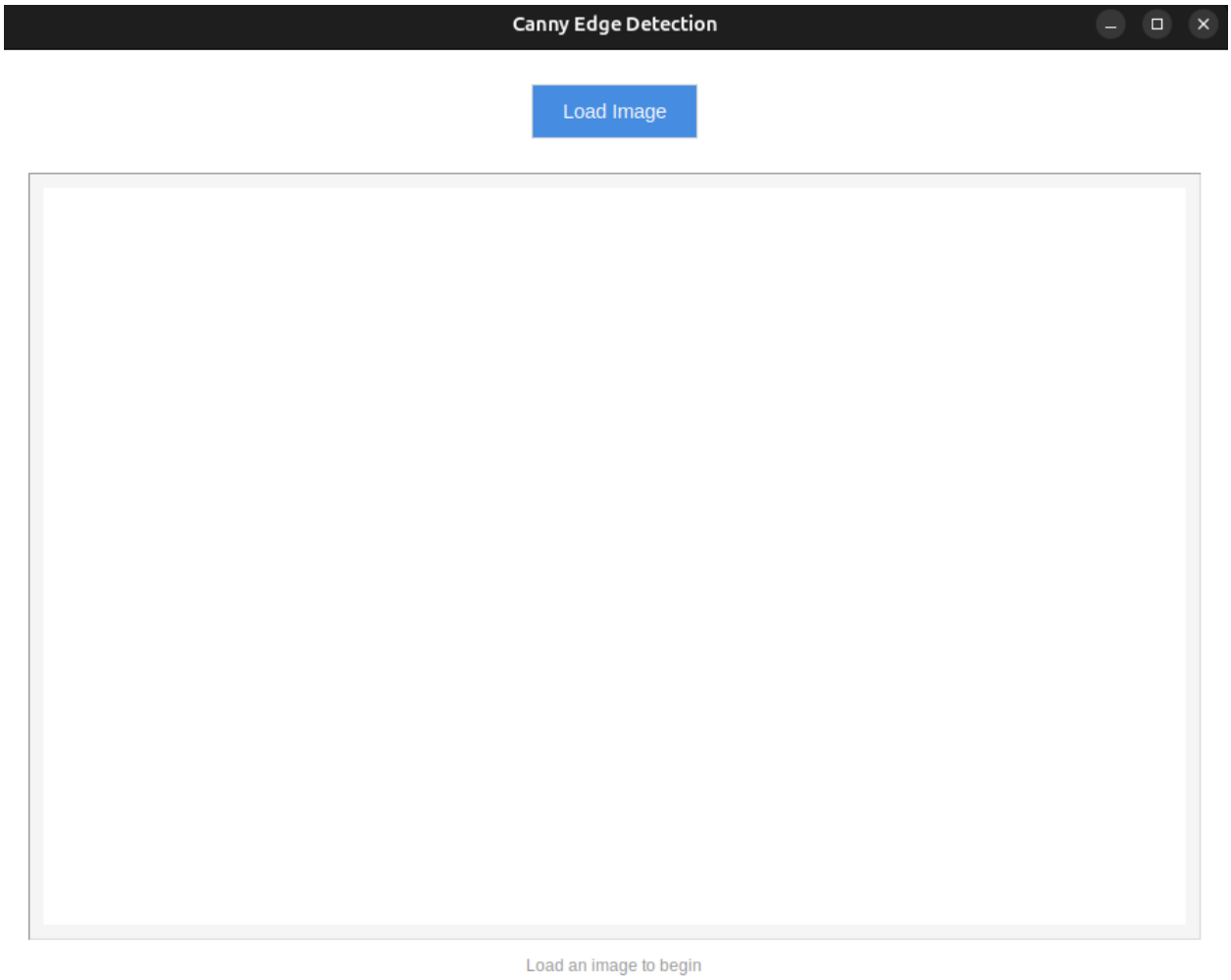
```
                                font=('Arial', 11),
                                bg='#4A90E2', fg='white',
                                relief=tk.FLAT, padx=20, pady=8)
        self.load_btn.pack(pady=5)

        self.apply_btn = tk.Button(self.control_frame, text="Apply Canny
Filter",
                                   command=self.apply_filter,
                                   font=('Arial', 11),
                                   bg='#7ED321', fg='white',
                                   relief=tk.FLAT, padx=20, pady=8)

        self.threshold_frame = tk.Frame(main_frame, bg='white')

        low_frame = tk.Frame(self.threshold_frame, bg='white')
        low_frame.pack(pady=5)
        tk.Label(low_frame, text="Low", font=('Arial', 10), bg='white',
fg='#666').pack(side=tk.LEFT)
        self.low_scale = tk.Scale(low_frame, from_=0, to=255,
                                  variable=self.low_threshold,
orient=tk.HORIZONTAL,
                                  command=self.update_filter,
                                  bg='white', fg='#333',
highlightthickness=0,
                                  troughcolor='#E8E8E8',
activebackground='#4A90E2',
                                  length=200, showvalue=0)
        self.low_scale.pack(side=tk.LEFT, padx=10)
        self.low_label = tk.Label(low_frame, text="50", font=('Arial', 10,
'bold'),
                                  bg='white', fg='#333', width=3)
        self.low_label.pack(side=tk.LEFT)

        high_frame = tk.Frame(self.threshold_frame, bg='white')
        high_frame.pack(pady=5)
        tk.Label(high_frame, text="High", font=('Arial', 10), bg='white',
fg='#666').pack(side=tk.LEFT)
        self.high_scale = tk.Scale(high_frame, from_=0, to=255,
                                   variable=self.high_threshold,
orient=tk.HORIZONTAL,
```

```
                                    command=self.update_filter,
                                    bg='white', fg='#333',
highlightthickness=0,
                                    troughcolor='#E8E8E8',
activebackground='#4A90E2',
                                    length=200, showvalue=0)
        self.high_scale.pack(side=tk.LEFT, padx=10)
        self.high_label = tk.Label(high_frame, text="150", font=('Arial',
10, 'bold'),
                                   bg='white', fg='#333', width=3)
        self.high_label.pack(side=tk.LEFT)

        self.action_frame = tk.Frame(main_frame, bg='white')

        self.reset_btn = tk.Button(self.action_frame, text="Reset",
                                   command=self.reset_values,
                                   font=('Arial', 9),
                                   bg='#F5A623', fg='white',
                                   relief=tk.FLAT, padx=15, pady=5)
        self.reset_btn.pack(side=tk.LEFT, padx=(0, 10))

        self.save_btn = tk.Button(self.action_frame, text="Save",
                                  command=self.save_image,
                                  font=('Arial', 9),
                                  bg='#50E3C2', fg='white',
                                  relief=tk.FLAT, padx=15, pady=5)
        self.save_btn.pack(side=tk.LEFT)

        self.image_frame = tk.Frame(main_frame, bg='#F8F8F8',
relief=tk.SUNKEN, bd=1)
        self.image_frame.pack(fill=tk.BOTH, expand=True)

        self.canvas = tk.Canvas(self.image_frame, bg='white',
highlightthickness=0)
        self.canvas.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        self.status_label = tk.Label(main_frame, text="Load an image to
begin",
                                     font=('Arial', 9), fg='#999',
bg='white')
```

```
        self.status_label.pack(pady=(10, 0))

    def load_image(self):
        downloads_path = os.path.join(os.path.expanduser("~"), "Downloads")
        file_path = filedialog.askopenfilename(
            initialdir=downloads_path,
            title="Select Image",
            filetypes=[("Images", "*.jpg *.jpeg *.png *.bmp *.tiff")]
        )
        if file_path:
            try:
                self.original_image = cv2.imread(file_path)
                if self.original_image is None:
                    raise ValueError("Could not load image")
                self.original_rgb = cv2.cvtColor(self.original_image,
cv2.COLOR_BGR2RGB)
                self.filter_applied = False
                self.processed_image = None
                self.threshold_frame.pack_forget()
                self.action_frame.pack_forget()
                self.apply_btn.pack(pady=5)
                self.display_image(self.original_rgb)
                filename = file_path.split('/')[-1]
                self.status_label.config(text=f"{filename} loaded")
            except Exception as e:
                messagebox.showerror("Error", f"Failed to load image:
{str(e)}")
                self.status_label.config(text="Error loading image")

    def apply_filter(self):
        if self.original_image is None:
            return
        self.apply_btn.pack_forget()
        self.threshold_frame.pack(pady=(0, 15))
        self.action_frame.pack(pady=(0, 15))
        self.filter_applied = True
        self.update_filter()
        self.status_label.config(text="Adjust thresholds to fine-tune
edges")
```

```
    def update_filter(self, event=None):
        if not self.filter_applied or self.original_image is None:
            return
        low_val = self.low_threshold.get()
        high_val = self.high_threshold.get()
        self.low_label.config(text=str(low_val))
        self.high_label.config(text=str(high_val))
        if low_val >= high_val:
            if event and event.widget == self.low_scale:
                self.high_threshold.set(min(255, low_val + 1))
            else:
                self.low_threshold.set(max(0, high_val - 1))
            return
        try:
            gray = cv2.cvtColor(self.original_image, cv2.COLOR_BGR2GRAY)
            blurred = cv2.GaussianBlur(gray, (5, 5), 0)
            edges = cv2.Canny(blurred, low_val, high_val)
            self.processed_image = edges
            self.display_image(edges)
        except Exception as e:
            messagebox.showerror("Error", f"Processing failed: {str(e)}")

    def display_image(self, image):
        if image is None:
            return
        self.canvas.update_idletasks()
        canvas_width = self.canvas.winfo_width()
        canvas_height = self.canvas.winfo_height()
        if canvas_width < 50 or canvas_height < 50:
            canvas_width, canvas_height = 600, 400
        if len(image.shape) == 3:
            h, w = image.shape[:2]
        else:
            h, w = image.shape
        scale = min((canvas_width-20)/w, (canvas_height-20)/h, 1.0)
        if scale < 1.0:
            new_w, new_h = int(w * scale), int(h * scale)
            display_img = cv2.resize(image, (new_w, new_h))
        else:
            display_img = image.copy()
```

```
        if len(display_img.shape) == 3:
            pil_img = Image.fromarray(display_img)
        else:
            pil_img = Image.fromarray(display_img, mode='L')
        self.photo = ImageTk.PhotoImage(pil_img)
        self.canvas.delete("all")
        x = (canvas_width - pil_img.width) // 2
        y = (canvas_height - pil_img.height) // 2
        self.canvas.create_image(x, y, anchor=tk.NW, image=self.photo)

    def reset_values(self):
        self.low_threshold.set(50)
        self.high_threshold.set(150)
        self.update_filter()

    def save_image(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No processed image to save")
            return
        file_path = filedialog.asksaveasfilename(
            title="Save Image",
            defaultextension=".png",
            filetypes=[("PNG", "*.png"), ("JPEG", "*.jpg")]
        )
        if file_path:
            try:
                cv2.imwrite(file_path, self.processed_image)
                messagebox.showinfo("Success", "Image saved!")
                self.status_label.config(text="Image saved successfully")
            except Exception as e:
                messagebox.showerror("Error", f"Save failed: {str(e)}")

def main():
    root = tk.Tk()
    app = MinimalCannyDetector(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```


Canny Edge Detection
Load Image
Load an image to begin

Canny Edge Detection

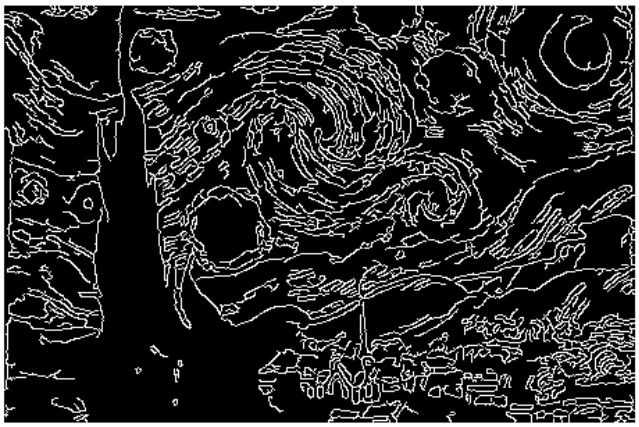
Load Image

Apply Canny Filter

download.jpeg loaded

Canny Edge Detection

Load Image

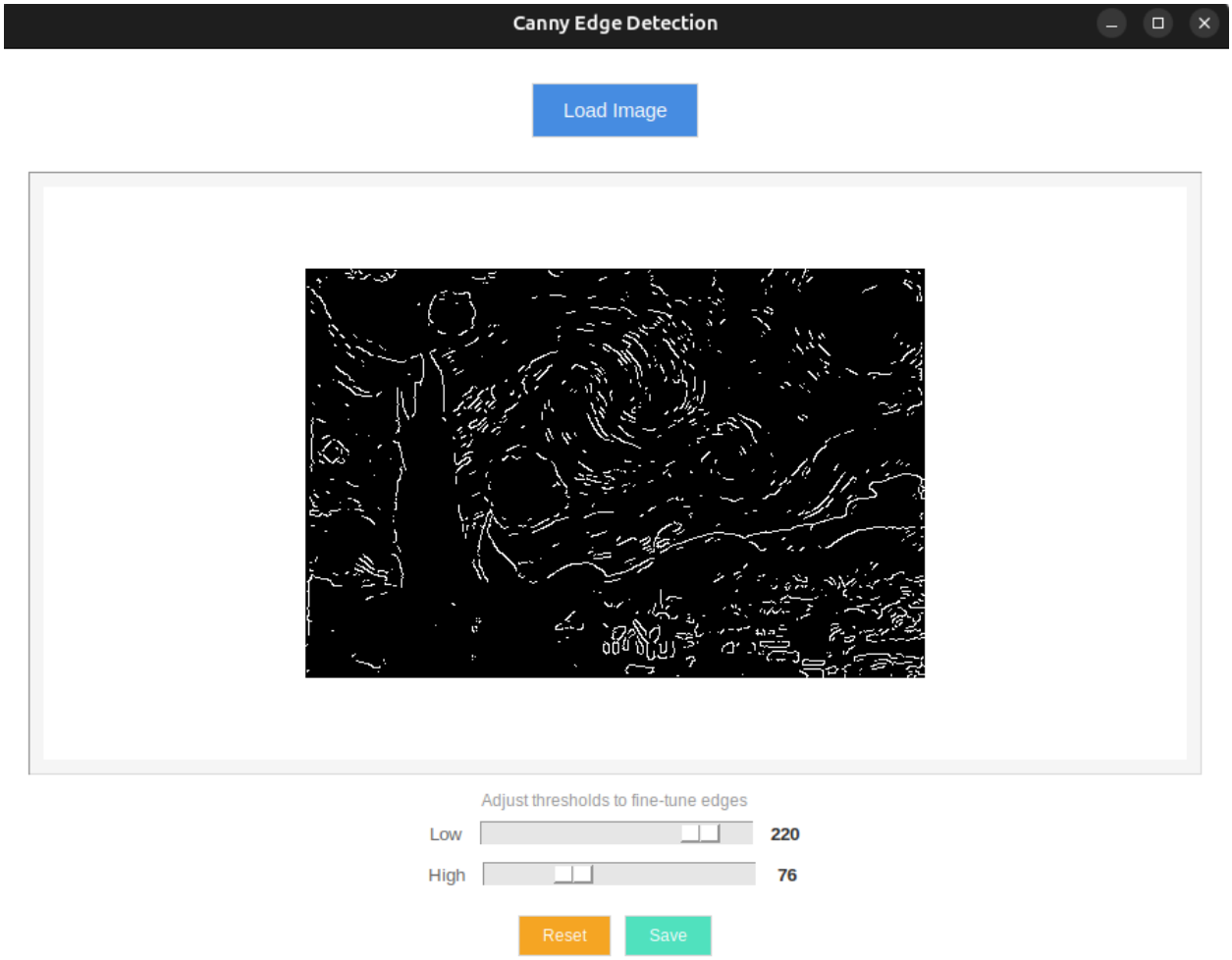


Adjust thresholds to fine-tune edges

Low 50

High 150

Reset Save



Q2. Create an application that detects and highlights objects of a specific color in a video stream.

```
import tkinter as tk
from tkinter import filedialog, colorchooser, messagebox
import cv2
import numpy as np
from PIL import Image, ImageTk
import os

class ColorObjectDetector:
    def __init__(self, root):
        self.root = root
        self.root.title("Color Object Detector")
        self.root.geometry("900x700")
```

```
        self.root.configure(bg='white')

        self.video_path = None
        self.cap = None
        self.color_lower = None
        self.color_upper = None
        self.running = False
        self.selected_color = None

        self.setup_ui()

    def setup_ui(self):
        control_frame = tk.Frame(self.root, bg='white')
        control_frame.pack(pady=10)

        tk.Button(control_frame, text="Load Video", bg="#5F7FA5",
fg="white",
                  font=("Arial", 11), relief=tk.FLAT, padx=20, pady=8,
                  command=self.load_video).pack(side=tk.LEFT, padx=5)

        tk.Button(control_frame, text="Choose Color", bg="#7ED321",
fg="white",
                  font=("Arial", 11), relief=tk.FLAT, padx=20, pady=8,
                  command=self.choose_color).pack(side=tk.LEFT, padx=5)

        tk.Button(control_frame, text="Start Detection", bg="#F5A623",
fg="white",
                  font=("Arial", 11), relief=tk.FLAT, padx=20, pady=8,
                  command=self.start_detection).pack(side=tk.LEFT, padx=5)

        tk.Button(control_frame, text="Stop", bg="#D0021B", fg="white",
                  font=("Arial", 11), relief=tk.FLAT, padx=20, pady=8,
                  command=self.stop_detection).pack(side=tk.LEFT, padx=5)

        self.status_label = tk.Label(self.root, text="Load a video and
choose a color to start",
                                     bg='white', font=("Arial", 10))
        self.status_label.pack(pady=5)

        self.canvas = tk.Canvas(self.root, bg='black')
```

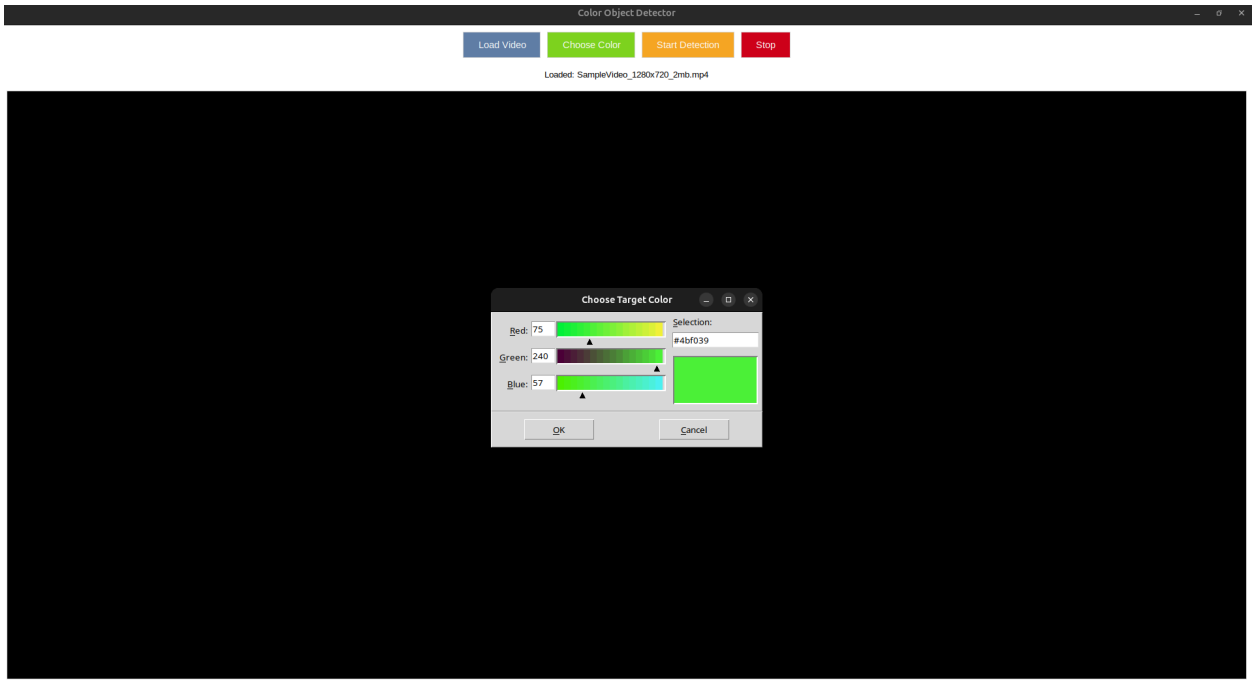
```
        self.canvas.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    def load_video(self):
        downloads_path = os.path.join(os.path.expanduser("~"), "Downloads")
        path = filedialog.askopenfilename(initialdir=downloads_path,
                                          title="Select Video",
                                          filetypes=[("Video files", "*.mp4
*.avi *.mov *.mkv")])
        if path:
            self.video_path = path
            if self.cap:
                self.cap.release()
            self.cap = cv2.VideoCapture(path)
            self.status_label.config(text=f"Loaded:
{os.path.basename(path)}")
            messagebox.showinfo("Video Loaded", f"Loaded:
{os.path.basename(path)}")

    def choose_color(self):
        color_code = colorchooser.askcolor(title="Choose Target Color")
        if color_code and color_code[0]:
            r, g, b = color_code[0]
            self.selected_color = (int(r), int(g), int(b))
            self.set_color_range(r, g, b)
            self.status_label.config(text=f"Color selected: R:{int(r)},
G:{int(g)}, B:{int(b)}")
            messagebox.showinfo("Color Selected", f"R:{int(r)}, G:{int(g)},
B:{int(b)}")

    def set_color_range(self, r, g, b):
        bgr_color = np.uint8([[[b, g, r]]])
        hsv_color = cv2.cvtColor(bgr_color, cv2.COLOR_BGR2HSV)[0][0]
        hue, sat, val = hsv_color

        if hue < 10 or hue > 170:
            if hue < 10:
                self.color_lower = np.array([0, 50, 50])
                self.color_upper = np.array([10, 255, 255])
                self.color_lower2 = np.array([170, 50, 50])
                self.color_upper2 = np.array([179, 255, 255])
```

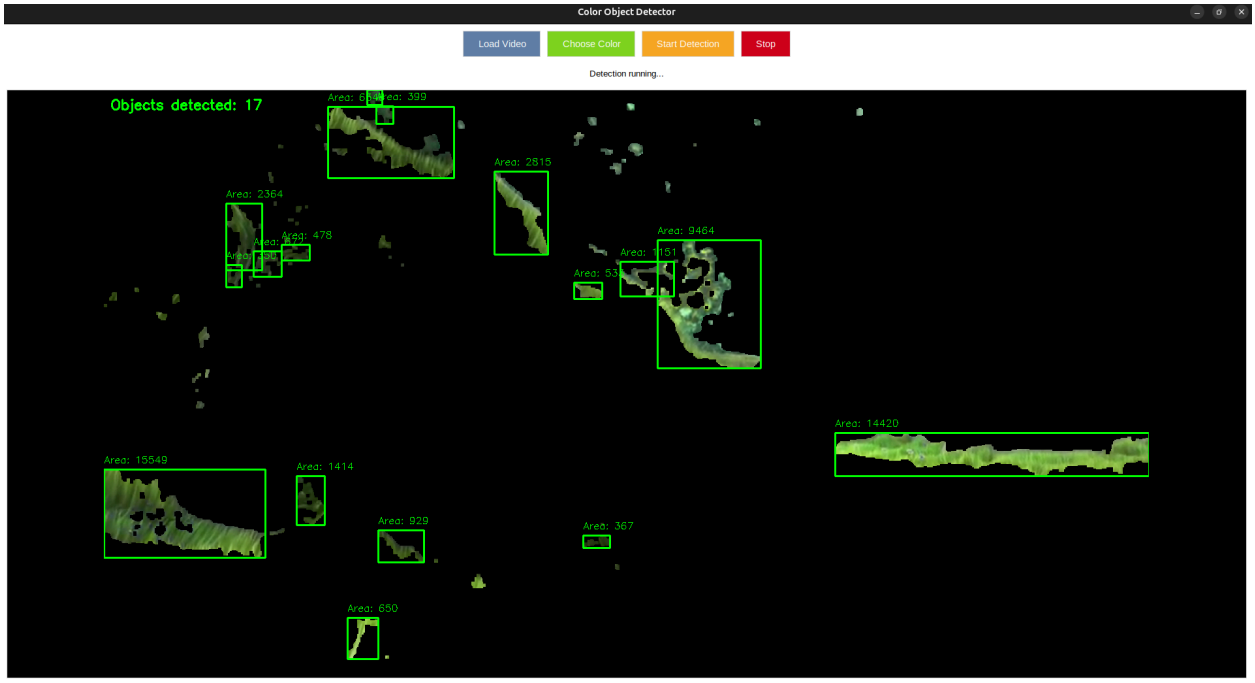
```
            else:
                self.color_lower = np.array([170, 50, 50])
                self.color_upper = np.array([179, 255, 255])
                self.color_lower2 = np.array([0, 50, 50])
                self.color_upper2 = np.array([10, 255, 255])
        else:
            hue_tolerance = 15
            self.color_lower = np.array([max(0, hue - hue_tolerance), 50,
50])
            self.color_upper = np.array([min(179, hue + hue_tolerance),
255, 255])
            self.color_lower2 = None
            self.color_upper2 = None

    def start_detection(self):
        if not self.cap or self.color_lower is None:
            messagebox.showerror("Error", "Please load a video and choose a
color first!")
            return
        self.running = True
        self.status_label.config(text="Detection running...")
        self.process_frame()

    def stop_detection(self):
        self.running = False
        self.status_label.config(text="Detection stopped")

    def create_mask(self, hsv):
        mask = cv2.inRange(hsv, self.color_lower, self.color_upper)

        if hasattr(self, 'color_lower2') and self.color_lower2 is not None:
            mask2 = cv2.inRange(hsv, self.color_lower2, self.color_upper2)
            mask = cv2.bitwise_or(mask, mask2)

        kernel = np.ones((5, 5), np.uint8)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

        return mask
```

```
    def resize_frame_to_canvas(self, frame):
        canvas_width = self.canvas.winfo_width()
        canvas_height = self.canvas.winfo_height()

        if canvas_width <= 1 or canvas_height <= 1:
            return frame

        frame_height, frame_width = frame.shape[:2]

        scale_w = canvas_width / frame_width
        scale_h = canvas_height / frame_height
        scale = min(scale_w, scale_h)

        new_width = int(frame_width * scale)
        new_height = int(frame_height * scale)

        resized_frame = cv2.resize(frame, (new_width, new_height))
        return resized_frame

    def process_frame(self):
        if not self.running or not self.cap:
            return

        ret, frame = self.cap.read()
        if not ret:
            self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
            ret, frame = self.cap.read()
            if not ret:
                self.running = False
                self.status_label.config(text="Error reading video")
                return

        frame = self.resize_frame_to_canvas(frame)

        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        mask = self.create_mask(hsv)

        result = cv2.bitwise_and(frame, frame, mask=mask)
```

```
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        object_count = 0

        for cnt in contours:
            area = cv2.contourArea(cnt)
            if area > 300:
                object_count += 1
                x, y, w, h = cv2.boundingRect(cnt)
                cv2.rectangle(result, (x, y), (x + w, y + h), (0, 255, 0),
2)
                cv2.putText(result, f'Area: {int(area)}', (x, y-10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

        cv2.putText(result, f'Objects detected: {object_count}', (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        frame_rgb = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
        img_pil = Image.fromarray(frame_rgb)
        img_tk = ImageTk.PhotoImage(image=img_pil)

        self.canvas.delete("all")
        canvas_width = self.canvas.winfo_width()
        canvas_height = self.canvas.winfo_height()

        x_offset = (canvas_width - img_pil.width) // 2
        y_offset = (canvas_height - img_pil.height) // 2

        self.canvas.create_image(max(0, x_offset), max(0, y_offset),
anchor=tk.NW, image=img_tk)
        self.canvas.image = img_tk

        if self.running:
            self.root.after(30, self.process_frame)

    def __del__(self):
        if self.cap:
            self.cap.release()

def main():
```

```
    root = tk.Tk()
    app = ColorObjectDetector(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```





Q3. Build a system that can detect and count the number of faces in an image or video

```
import tkinter as tk
from tkinter import filedialog, messagebox
import cv2
import numpy as np
```

```
from PIL import Image, ImageTk
import os

class FaceDetector:
    def __init__(self, root):
        self.root = root
        self.root.title("Face Detection and Counting System")
        self.root.geometry("1000x800")
        self.root.configure(bg='white')

        self.current_file = None
        self.cap = None
        self.running = False
        self.is_video = False
        self.load_face_detector()

        self.setup_ui()

    def load_face_detector(self):
        try:
            self.face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades
+ 'haarcascade_frontalface_default.xml')
        except Exception as e:
            messagebox.showerror("Error", f"Failed to load face detection
model: {str(e)}")

    def setup_ui(self):
        control_frame = tk.Frame(self.root, bg='white', relief=tk.RAISED,
bd=1)
        control_frame.pack(fill=tk.X, padx=5, pady=5)
        file_frame = tk.Frame(control_frame, bg='white')
        file_frame.pack(side=tk.LEFT, padx=5)
        tk.Button(file_frame, text="Load Image", bg="#4CAF50", fg="white",
                  font=("Arial", 10, "bold"), relief=tk.FLAT, padx=15,
pady=5,
                  command=self.load_image).pack(side=tk.LEFT, padx=2)
        tk.Button(file_frame, text="Load Video", bg="#2196F3", fg="white",
                  font=("Arial", 10, "bold"), relief=tk.FLAT, padx=15,
pady=5,
                  command=self.load_video).pack(side=tk.LEFT, padx=2)
```

```
        tk.Button(file_frame, text="Use Webcam", bg="#FF9800", fg="white",
                  font=("Arial", 10, "bold"), relief=tk.FLAT, padx=15,
pady=5,
                  command=self.use_webcam).pack(side=tk.LEFT, padx=2)
        detection_frame = tk.Frame(control_frame, bg='white')
        detection_frame.pack(side=tk.LEFT, padx=20)
        tk.Button(detection_frame, text="Detect Faces", bg="#9C27B0",
fg="white",
                  font=("Arial", 10, "bold"), relief=tk.FLAT, padx=15,
pady=5,
                  command=self.detect_faces).pack(side=tk.LEFT, padx=5)
        video_frame = tk.Frame(control_frame, bg='white')
        video_frame.pack(side=tk.LEFT, padx=20)
        tk.Button(video_frame, text="Start Video", bg="#4CAF50",
fg="white",
                  font=("Arial", 10, "bold"), relief=tk.FLAT, padx=15,
pady=5,
                  command=self.start_video_detection).pack(side=tk.LEFT,
padx=2)
        tk.Button(video_frame, text="Stop", bg="#F44336", fg="white",
                  font=("Arial", 10, "bold"), relief=tk.FLAT, padx=15,
pady=5,
                  command=self.stop_detection).pack(side=tk.LEFT, padx=2)
        status_frame = tk.Frame(self.root, bg='white')
        status_frame.pack(fill=tk.X, padx=5, pady=2)
        self.status_label = tk.Label(status_frame, text="Load an image or
video to start face detection",
                                     bg='white', font=("Arial", 10),
fg='blue')
        self.status_label.pack(side=tk.LEFT)
        self.face_count_label = tk.Label(status_frame, text="Faces: 0",
                                         bg='white', font=("Arial", 12,
"bold"), fg='red')
        self.face_count_label.pack(side=tk.RIGHT, padx=20)
        self.canvas = tk.Canvas(self.root, bg='black', relief=tk.SUNKEN,
bd=2)
        self.canvas.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
        self.canvas.bind('<Configure>', self.on_canvas_resize)

    def on_canvas_resize(self, event):
```

```
        if hasattr(self, 'current_image') and self.current_image is not
None and not self.running:
            self.display_image(self.current_image)

    def load_image(self):
        file_types = [
            ("Image files", "*.jpg *.jpeg *.png *.bmp *.tiff *.gif"),
            ("JPEG files", "*.jpg *.jpeg"),
            ("PNG files", "*.png"),
            ("All files", "*.*")
        ]

        file_path = filedialog.askopenfilename(
            title="Select Image File",
            filetypes=file_types,
            initialdir=os.path.join(os.path.expanduser("~"), "Downloads")
        )

        if file_path:
            try:
                self.current_file = file_path
                self.is_video = False
                image = cv2.imread(file_path)
                if image is None:
                    raise ValueError("Could not load image")

                self.current_image = image.copy()
                self.display_image(image)
                self.status_label.config(text=f"Loaded:
{os.path.basename(file_path)}")
                self.face_count_label.config(text="Faces: 0")

            except Exception as e:
                messagebox.showerror("Error", f"Failed to load image:
{str(e)}")

    def load_video(self):
        file_types = [
            ("Video files", "*.mp4 *.avi *.mov *.mkv *.wmv *.flv"),
            ("MP4 files", "*.mp4"),
```

```
            ("AVI files", "*.avi"),
            ("All files", "*.*")
        ]

        file_path = filedialog.askopenfilename(
            title="Select Video File",
            filetypes=file_types,
            initialdir=os.path.join(os.path.expanduser("~"), "Downloads")
        )

        if file_path:
            try:
                if self.cap:
                    self.cap.release()

                self.cap = cv2.VideoCapture(file_path)
                if not self.cap.isOpened():
                    raise ValueError("Could not open video")

                self.current_file = file_path
                self.is_video = True
                self.status_label.config(text=f"Loaded:
{os.path.basename(file_path)}")
                self.face_count_label.config(text="Faces: 0")

                ret, frame = self.cap.read()
                if ret:
                    self.current_image = frame.copy()
                    self.display_image(frame)
                    self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

            except Exception as e:
                messagebox.showerror("Error", f"Failed to load video:
{str(e)}")

    def use_webcam(self):
        try:
            if self.cap:
                self.cap.release()
```

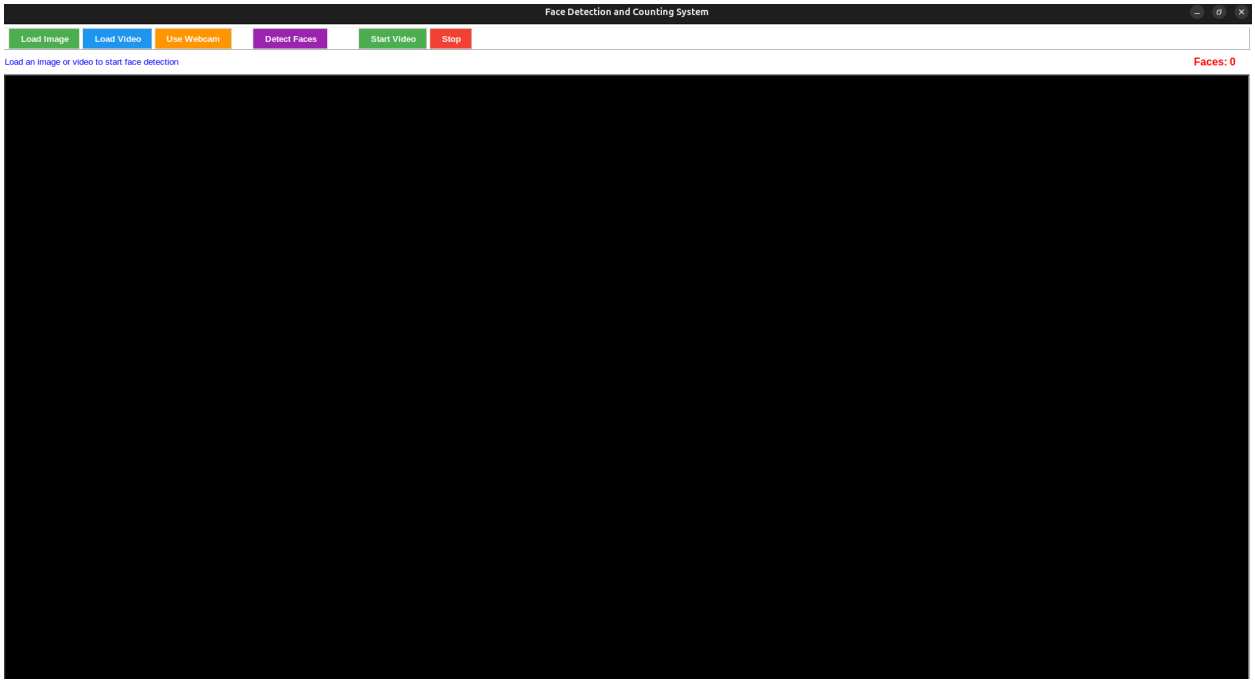
```
            self.cap = cv2.VideoCapture(0)
            if not self.cap.isOpened():
                raise ValueError("Could not access webcam")

            self.current_file = "Webcam"
            self.is_video = True
            self.status_label.config(text="Webcam ready")
            self.face_count_label.config(text="Faces: 0")

        except Exception as e:
            messagebox.showerror("Error", f"Failed to access webcam:
{str(e)}")

    def detect_faces_in_frame(self, frame):
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = self.face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(80, 80),
            flags=cv2.CASCADE_SCALE_IMAGE
        )

        return faces

    def detect_faces(self):
        if not hasattr(self, 'current_image') or self.current_image is
None:
            messagebox.showwarning("Warning", "Please load an image
first!")
            return

        try:
            frame = self.current_image.copy()
            faces = self.detect_faces_in_frame(frame)
            for i, (x, y, w, h) in enumerate(faces):
                cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0),
2)
                cv2.putText(frame, f'Face {i+1}', (x, y - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```

```
            cv2.putText(frame, f'Total Faces: {len(faces)}', (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
            self.display_image(frame)
            self.face_count_label.config(text=f"Faces: {len(faces)}")
            self.status_label.config(text=f"Detection complete - Found
{len(faces)} face(s)")

        except Exception as e:
            messagebox.showerror("Error", f"Face detection failed:
{str(e)}")

    def start_video_detection(self):
        if not self.cap:
            messagebox.showwarning("Warning", "Please load a video or start
webcam first!")
            return

        self.running = True
        self.status_label.config(text="Running face detection...")
        self.process_video_frame()

    def stop_detection(self):
        self.running = False
        self.status_label.config(text="Detection stopped")

    def process_video_frame(self):
        if not self.running or not self.cap:
            return

        ret, frame = self.cap.read()
        if not ret:
            if self.current_file != "Webcam":
                self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
                ret, frame = self.cap.read()

            if not ret:
                self.running = False
                self.status_label.config(text="Video ended or webcam
disconnected")
                return
```

```
        try:
            faces = self.detect_faces_in_frame(frame)
            for i, (x, y, w, h) in enumerate(faces):
                cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0),
2)
                cv2.putText(frame, f'Face {i+1}', (x, y - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
            cv2.putText(frame, f'Faces: {len(faces)}', (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

            self.display_image(frame)
            self.face_count_label.config(text=f"Faces: {len(faces)}")

        except Exception as e:
            print(f"Error processing frame: {str(e)}")

        # Schedule next frame
        if self.running:
            self.root.after(30, self.process_video_frame)

    def resize_image_to_canvas(self, image):
        """Resize image to fit canvas while maintaining aspect ratio"""
        canvas_width = self.canvas.winfo_width()
        canvas_height = self.canvas.winfo_height()

        if canvas_width <= 1 or canvas_height <= 1:
            return image

        img_height, img_width = image.shape[:2]

        # Calculate scaling factor
        scale_w = canvas_width / img_width
        scale_h = canvas_height / img_height
        scale = min(scale_w, scale_h)

        # Calculate new dimensions
        new_width = int(img_width * scale)
        new_height = int(img_height * scale)
```

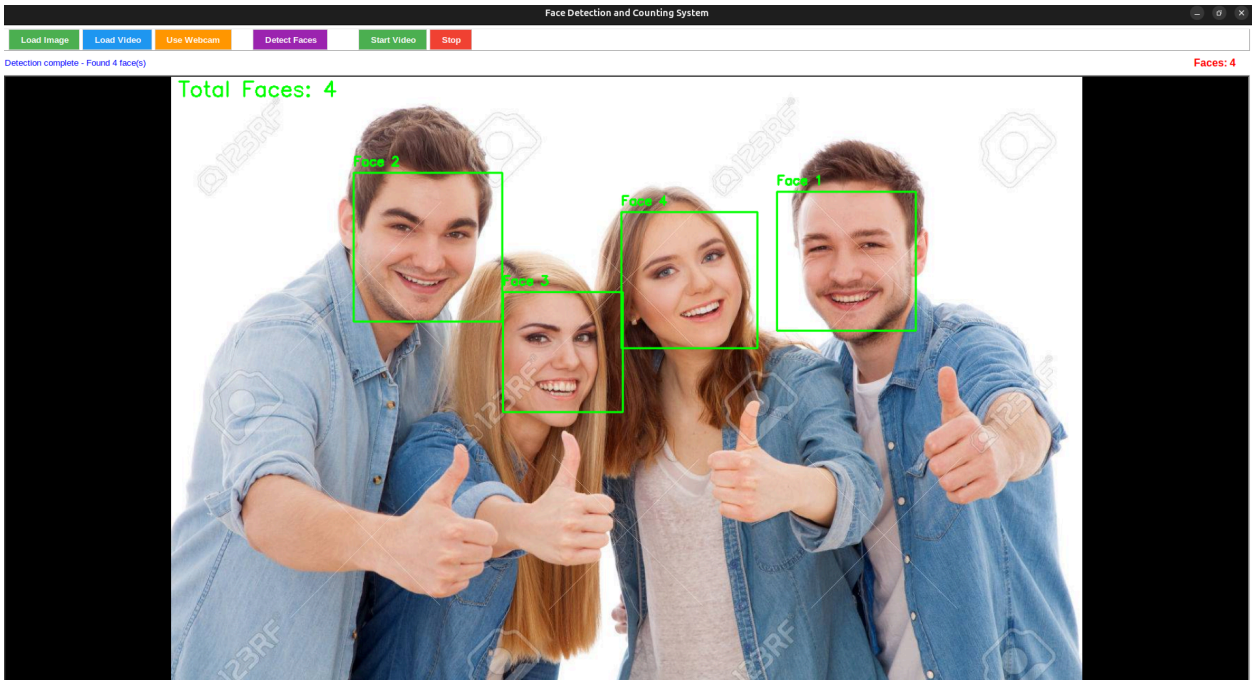
```
        # Resize image
        resized_image = cv2.resize(image, (new_width, new_height))
        return resized_image

    def display_image(self, image):
        """Display image on canvas"""
        try:
            # Resize image to fit canvas
            display_image = self.resize_image_to_canvas(image)

            # Convert BGR to RGB
            image_rgb = cv2.cvtColor(display_image, cv2.COLOR_BGR2RGB)

            # Convert to PIL Image and then to PhotoImage
            pil_image = Image.fromarray(image_rgb)
            photo = ImageTk.PhotoImage(pil_image)

            # Clear canvas and display image
            self.canvas.delete("all")

            # Center image on canvas
            canvas_width = self.canvas.winfo_width()
            canvas_height = self.canvas.winfo_height()
            x_offset = (canvas_width - pil_image.width) // 2
            y_offset = (canvas_height - pil_image.height) // 2

            self.canvas.create_image(max(0, x_offset), max(0, y_offset),
                                     anchor=tk.NW, image=photo)
            self.canvas.image = photo  # Keep a reference

        except Exception as e:
            print(f"Error displaying image: {str(e)}")

    def __del__(self):
        """Cleanup resources"""
        if self.cap:
            self.cap.release()

def main():
    root = tk.Tk()
```

```
    app = FaceDetector(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

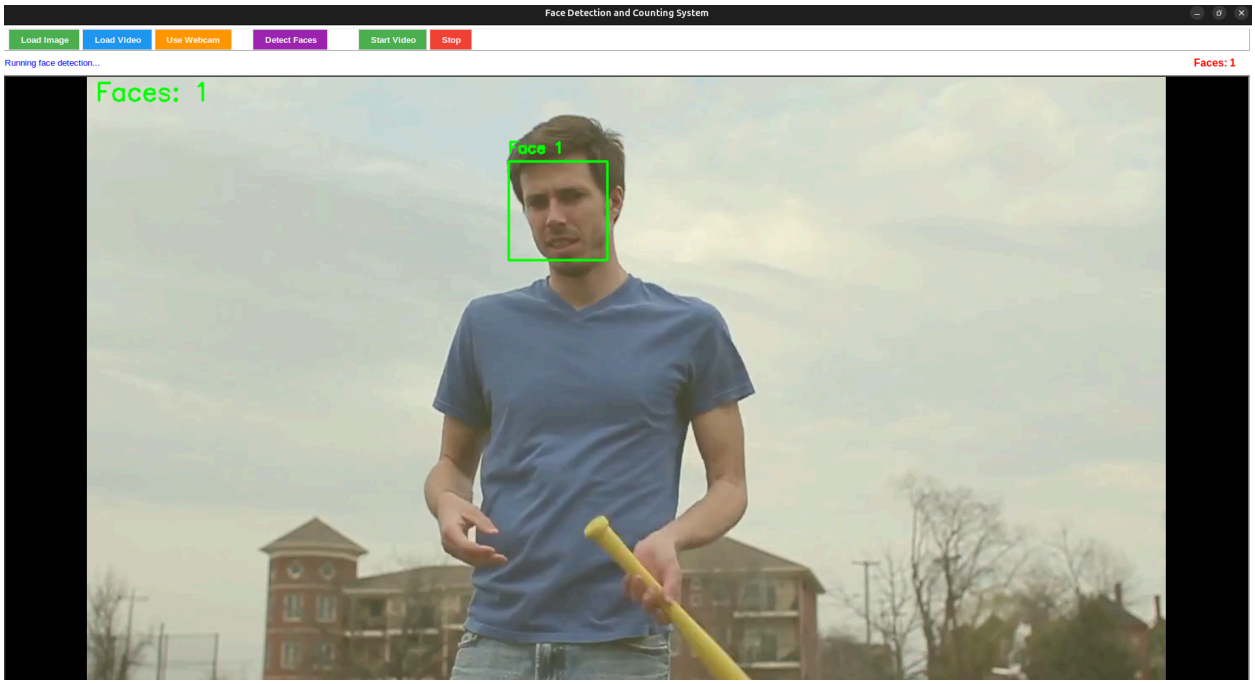


Image


Face Detection and Counting System
Load Image
Load Video
Use Webcam
Detect Faces
Start Video
Stop
Loaded: 38738606-group-of-happy-young-people-isolated-on-white-background.jpg
Faces: 0


Face Detection and Counting System
Load Image
Load Video
Use Webcam
Detect Faces
Start Video
Stop
Detection complete - Found 4 face(s)
Faces: 4
Total Faces: 4
Face 2
Face 3
Face 4
Face 1

Video



Q4. Implement a program that can resize an image, rotate it by a certain angle, or blur it using a specified kernel.

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
from PIL import Image, ImageTk, ImageFilter, ImageEnhance
import numpy as np
from scipy import ndimage
import os

class ImageProcessor:
    def __init__(self, root):
        self.root = root
        self.root.title("Image Processing Tool")
        self.root.geometry("1600x1000")

        self.original_image = None
        self.processed_image = None
        self.display_image = None
```

```
        self.setup_ui()

    def setup_ui(self):
        main_frame = ttk.Frame(self.root, padding="10")
        main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        self.root.columnconfigure(0, weight=1)
        self.root.rowconfigure(0, weight=1)
        main_frame.columnconfigure(1, weight=6)
        main_frame.rowconfigure(0, weight=1)

        control_frame = ttk.LabelFrame(main_frame, text="Controls",
padding="8", width=280)
        control_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N,
tk.S), padx=(0, 8))
        control_frame.grid_propagate(False)

        file_frame = ttk.LabelFrame(control_frame, text="File Operations",
padding="5")
        file_frame.pack(fill="x", pady=(0, 10))

        ttk.Button(file_frame, text="Load Image",
command=self.load_image).pack(fill="x", pady=2)
        ttk.Button(file_frame, text="Save Image",
command=self.save_image).pack(fill="x", pady=2)
        ttk.Button(file_frame, text="Reset to Original",
command=self.reset_image).pack(fill="x", pady=2)

        resize_frame = ttk.LabelFrame(control_frame, text="Resize",
padding="5")
        resize_frame.pack(fill="x", pady=(0, 10))

        ttk.Label(resize_frame, text="Width:").pack(anchor="w")
        self.width_var = tk.StringVar(value="400")
        ttk.Entry(resize_frame, textvariable=self.width_var,
width=10).pack(fill="x", pady=2)

        ttk.Label(resize_frame, text="Height:").pack(anchor="w")
        self.height_var = tk.StringVar(value="300")
```

```
        ttk.Entry(resize_frame, textvariable=self.height_var,
width=10).pack(fill="x", pady=2)

        self.maintain_aspect = tk.BooleanVar(value=True)
        ttk.Checkbutton(resize_frame, text="Maintain Aspect Ratio",
                        variable=self.maintain_aspect).pack(anchor="w",
pady=2)

        ttk.Button(resize_frame, text="Resize Image",
command=self.resize_image).pack(fill="x", pady=2)

        rotate_frame = ttk.LabelFrame(control_frame, text="Rotation",
padding="5")
        rotate_frame.pack(fill="x", pady=(0, 10))

        ttk.Label(rotate_frame, text="Angle (degrees):").pack(anchor="w")
        self.angle_var = tk.StringVar(value="0")
        ttk.Entry(rotate_frame, textvariable=self.angle_var,
width=10).pack(fill="x", pady=2)

        angle_buttons_frame = ttk.Frame(rotate_frame)
        angle_buttons_frame.pack(fill="x", pady=2)

        ttk.Button(angle_buttons_frame, text="90°", width=5,
                   command=lambda: self.set_angle(90)).pack(side="left",
padx=1)
        ttk.Button(angle_buttons_frame, text="180°", width=5,
                   command=lambda: self.set_angle(180)).pack(side="left",
padx=1)
        ttk.Button(angle_buttons_frame, text="270°", width=5,
                   command=lambda: self.set_angle(270)).pack(side="left",
padx=1)

        ttk.Button(rotate_frame, text="Rotate Image",
command=self.rotate_image).pack(fill="x", pady=2)

        blur_frame = ttk.LabelFrame(control_frame, text="Blur",
padding="5")
        blur_frame.pack(fill="x", pady=(0, 10))
```

```
        ttk.Label(blur_frame, text="Blur Type:").pack(anchor="w")
        self.blur_type = tk.StringVar(value="Gaussian")
        blur_combo = ttk.Combobox(blur_frame, textvariable=self.blur_type,
                                  values=["Gaussian", "Box", "Motion"],
state="readonly")
        blur_combo.pack(fill="x", pady=2)

        ttk.Label(blur_frame, text="Intensity:").pack(anchor="w")
        self.blur_intensity = tk.DoubleVar(value=2.0)
        blur_scale = ttk.Scale(blur_frame, from_=0.1, to=10.0,
                               variable=self.blur_intensity,
orient="horizontal")
        blur_scale.pack(fill="x", pady=2)

        self.blur_label = ttk.Label(blur_frame, text="2.0")
        self.blur_label.pack(anchor="w")
        blur_scale.configure(command=self.update_blur_label)

        ttk.Button(blur_frame, text="Apply Blur",
command=self.blur_image).pack(fill="x", pady=2)
        display_frame = ttk.LabelFrame(main_frame, text="Image Display",
padding="5")
        display_frame.grid(row=0, column=1, sticky=(tk.W, tk.E, tk.N,
tk.S))
        display_frame.columnconfigure(0, weight=1)
        display_frame.rowconfigure(0, weight=1)

        self.canvas = tk.Canvas(display_frame, bg="white", relief="sunken",
bd=1,
                                highlightthickness=0)
        self.canvas.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

        v_scrollbar = ttk.Scrollbar(display_frame, orient="vertical",
command=self.canvas.yview)
        v_scrollbar.grid(row=0, column=1, sticky=(tk.N, tk.S))
        self.canvas.configure(yscrollcommand=v_scrollbar.set)

        h_scrollbar = ttk.Scrollbar(display_frame, orient="horizontal",
command=self.canvas.xview)
        h_scrollbar.grid(row=1, column=0, sticky=(tk.W, tk.E))
```

```
        self.canvas.configure(xscrollcommand=h_scrollbar.set)

        self.status_var = tk.StringVar(value="Ready - Please load an
image")
        status_bar = ttk.Label(main_frame, textvariable=self.status_var,
relief="sunken")
        status_bar.grid(row=1, column=0, columnspan=2, sticky=(tk.W, tk.E),
pady=(10, 0))

    def update_filter_intensity_label(self, value):
        self.filter_intensity_label.config(text=f"{float(value):.1f}")

    def on_filter_select(self, event=None):
        if self.filter_type.get() == "Custom":
            self.custom_kernel_frame.pack(fill="x", pady=(5, 0))
        else:
            self.custom_kernel_frame.pack_forget()

    def get_predefined_kernel(self, filter_name):
        kernels = {
            "Sharpen": np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]]),
            "Edge Detection": np.array([[-1, -1, -1], [-1, 8, -1], [-1, -1,
-1]]),
            "Emboss": np.array([[-2, -1, 0], [-1, 1, 1], [0, 1, 2]]),
            "High Pass": np.array([[-1, -1, -1], [-1, 9, -1], [-1, -1,
-1]]),
            "Gaussian Sharpen": np.array([[0, -1, 0], [-1, 6, -1], [0, -1,
0]]) / 2,
            "Unsharp Mask": np.array([[-1, -4, -1], [-4, 20, -4], [-1, -4,
-1]]) / 8
        }
        return kernels.get(filter_name)

    def apply_filter(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image loaded")
            return

        filter_name = self.filter_type.get()
```

```
        if filter_name == "None":
            return
        elif filter_name == "Custom":
            self.apply_custom_kernel()
            return

        try:
            kernel = self.get_predefined_kernel(filter_name)
            if kernel is None:
                messagebox.showerror("Error", f"Unknown filter:
{filter_name}")
                return

            intensity = self.filter_intensity.get()
            if filter_name in ["Sharpen", "Gaussian Sharpen", "High Pass"]:
                center = kernel.shape[0] // 2
                original_center = kernel[center, center]
                kernel = kernel.copy()
                kernel[center, center] = (original_center - 1) * intensity
+ 1
            elif filter_name == "Unsharp Mask":
                kernel = kernel * intensity

            img_array = np.array(self.processed_image)

            if len(img_array.shape) == 3:
                for i in range(img_array.shape[2]):
                    img_array[:, :, i] = ndimage.convolve(img_array[:, :,
i], kernel, mode='reflect')
            else:
                img_array = ndimage.convolve(img_array, kernel,
mode='reflect')

            img_array = np.clip(img_array, 0, 255)

            self.processed_image = Image.fromarray(np.uint8(img_array))
            self.display_image_on_canvas()
            self.status_var.set(f"Applied {filter_name} filter (intensity:
{intensity:.1f})")
```

```
        except Exception as e:
            messagebox.showerror("Error", f"Failed to apply filter:
{str(e)}")

    def set_angle(self, angle):
        self.angle_var.set(str(angle))

    def load_image(self):
        file_path = filedialog.askopenfilename(
            title="Select Image",
            filetypes=[
                ("Image files", "*.jpg *.jpeg *.png *.bmp *.gif *.tiff"),
                ("All files", "*.*")
            ]
        )

        if file_path:
            try:
                self.original_image = Image.open(file_path)
                self.processed_image = self.original_image.copy()
                self.display_image_on_canvas()

                # Update size fields with current image size
                self.width_var.set(str(self.original_image.width))
                self.height_var.set(str(self.original_image.height))

                self.status_var.set(f"Loaded: {os.path.basename(file_path)}
"

f"({self.original_image.width}x{self.original_image.height})")

            except Exception as e:
                messagebox.showerror("Error", f"Failed to load image:
{str(e)}")

    def save_image(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image to save")
            return
```

```
        file_path = filedialog.asksaveasfilename(
            title="Save Image",
            defaultextension=".png",
            filetypes=[
                ("PNG files", "*.png"),
                ("JPEG files", "*.jpg"),
                ("All files", "*.*")
            ]
        )

        if file_path:
            try:
                self.processed_image.save(file_path)
                self.status_var.set(f"Saved:
{os.path.basename(file_path)}")
            except Exception as e:
                messagebox.showerror("Error", f"Failed to save image:
{str(e)}")

    def reset_image(self):
        if self.original_image is None:
            messagebox.showwarning("Warning", "No original image loaded")
            return

        self.processed_image = self.original_image.copy()
        self.display_image_on_canvas()
        self.status_var.set("Image reset to original")

    def display_image_on_canvas(self):
        if self.processed_image is None:
            return

        self.display_image = ImageTk.PhotoImage(self.processed_image)

        self.canvas.delete("all")
        self.canvas.create_image(0, 0, anchor="nw",
image=self.display_image)

        self.canvas.configure(scrollregion=self.canvas.bbox("all"))
```

```
    def resize_image(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image loaded")
            return

        try:
            width = int(self.width_var.get())
            height = int(self.height_var.get())

            if width <= 0 or height <= 0:
                raise ValueError("Width and height must be positive")

            if self.maintain_aspect.get():
                original_ratio = self.processed_image.width /
self.processed_image.height
                new_ratio = width / height

                if new_ratio > original_ratio:
                    width = int(height * original_ratio)
                else:
                    height = int(width / original_ratio)

                self.width_var.set(str(width))
                self.height_var.set(str(height))

            self.processed_image = self.processed_image.resize((width,
height), Image.Resampling.LANCZOS)
            self.display_image_on_canvas()
            self.status_var.set(f"Resized to {width}x{height}")

        except ValueError as e:
            messagebox.showerror("Error", f"Invalid size values: {str(e)}")
        except Exception as e:
            messagebox.showerror("Error", f"Failed to resize image:
{str(e)}")

    def rotate_image(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image loaded")
            return
```

```
        try:
            angle = float(self.angle_var.get())
            self.processed_image = self.processed_image.rotate(angle,
expand=True, fillcolor='white')
            self.display_image_on_canvas()
            self.status_var.set(f"Rotated by {angle} degrees")

        except ValueError:
            messagebox.showerror("Error", "Invalid angle value")
        except Exception as e:
            messagebox.showerror("Error", f"Failed to rotate image:
{str(e)}")

    def blur_image(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image loaded")
            return

        try:
            blur_type = self.blur_type.get()
            intensity = self.blur_intensity.get()

            if blur_type == "Gaussian":
                self.processed_image =
self.processed_image.filter(ImageFilter.GaussianBlur(radius=intensity))
            elif blur_type == "Box":
                self.processed_image =
self.processed_image.filter(ImageFilter.BoxBlur(radius=intensity))
            elif blur_type == "Motion":
                kernel_size = int(intensity * 2) + 1
                kernel = np.zeros((kernel_size, kernel_size))
                kernel[kernel_size//2, :] = 1.0
                kernel = kernel / kernel.sum()

                img_array = np.array(self.processed_image)

                if len(img_array.shape) == 3:
                    for i in range(img_array.shape[2]):
```

```
                    img_array[:, :, i] = ndimage.convolve(img_array[:,
:, i], kernel, mode='reflect')
                else:
                    img_array = ndimage.convolve(img_array, kernel,
mode='reflect')

                self.processed_image = Image.fromarray(np.uint8(img_array))

            self.display_image_on_canvas()
            self.status_var.set(f"Applied {blur_type} blur (intensity:
{intensity:.1f})")

        except Exception as e:
            messagebox.showerror("Error", f"Failed to apply blur:
{str(e)}")

    def update_blur_label(self, value):
        self.blur_label.config(text=f"{float(value):.1f}")

    def update_filter_intensity_label(self, value):
        self.filter_intensity_label.config(text=f"{float(value):.1f}")

    def on_filter_select(self, event=None):
        if self.filter_type.get() == "Custom":
            self.custom_kernel_frame.pack(fill="x", pady=(5, 0))
        else:
            self.custom_kernel_frame.pack_forget()

    def get_predefined_kernel(self, filter_name):
        kernels = {
            "Sharpen": np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]]),
            "Edge Detection": np.array([[-1, -1, -1], [-1, 8, -1], [-1, -1,
-1]]),
            "Emboss": np.array([[-2, -1, 0], [-1, 1, 1], [0, 1, 2]]),
            "High Pass": np.array([[-1, -1, -1], [-1, 9, -1], [-1, -1,
-1]]),
            "Gaussian Sharpen": np.array([[0, -1, 0], [-1, 6, -1], [0, -1,
0]]) / 2,
            "Unsharp Mask": np.array([[-1, -4, -1], [-4, 20, -4], [-1, -4,
-1]]) / 8
```

```
        }
        return kernels.get(filter_name)

    def apply_filter(self):
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image loaded")
            return

        filter_name = self.filter_type.get()

        if filter_name == "None":
            return
        elif filter_name == "Custom":
            self.apply_custom_kernel()
            return

        try:
            kernel = self.get_predefined_kernel(filter_name)
            if kernel is None:
                messagebox.showerror("Error", f"Unknown filter:
{filter_name}")
                return

            intensity = self.filter_intensity.get()
            if filter_name in ["Sharpen", "Gaussian Sharpen", "High Pass"]:
                center = kernel.shape[0] // 2
                original_center = kernel[center, center]
                kernel = kernel.copy()
                kernel[center, center] = (original_center - 1) * intensity
+ 1
            elif filter_name == "Unsharp Mask":
                kernel = kernel * intensity

            img_array = np.array(self.processed_image)

            if len(img_array.shape) == 3:
                for i in range(img_array.shape[2]):
                    img_array[:, :, i] = ndimage.convolve(img_array[:, :,
i], kernel, mode='reflect')
            else:
```

```
                img_array = ndimage.convolve(img_array, kernel,
mode='reflect')

            img_array = np.clip(img_array, 0, 255)

            self.processed_image = Image.fromarray(np.uint8(img_array))
            self.display_image_on_canvas()
            self.status_var.set(f"Applied {filter_name} filter (intensity:
{intensity:.1f})")

        except Exception as e:
            messagebox.showerror("Error", f"Failed to apply filter:
{str(e)}")
        if self.processed_image is None:
            messagebox.showwarning("Warning", "No image loaded")
            return

        try:
            kernel_text = self.kernel_text.get("1.0", tk.END).strip()
            lines = kernel_text.split('\n')

            kernel = []
            for line in lines:
                if line.strip():
                    row = [float(x) for x in line.split()]
                    kernel.append(row)

            kernel = np.array(kernel)

            if kernel.shape[0] != kernel.shape[1]:
                raise ValueError("Kernel must be square")

            if kernel.shape[0] % 2 == 0:
                raise ValueError("Kernel size must be odd")

            img_array = np.array(self.processed_image)

            if len(img_array.shape) == 3:
                for i in range(img_array.shape[2]):
```

```
                    img_array[:, :, i] = ndimage.convolve(img_array[:, :,
i], kernel, mode='reflect')
            else:
                img_array = ndimage.convolve(img_array, kernel,
mode='reflect')

            img_array = np.clip(img_array, 0, 255)

            self.processed_image = Image.fromarray(np.uint8(img_array))
            self.display_image_on_canvas()
            self.status_var.set(f"Applied custom kernel
({kernel.shape[0]}x{kernel.shape[1]})")

        except Exception as e:
            messagebox.showerror("Error", f"Failed to apply custom kernel:
{str(e)}")

def main():
    root = tk.Tk()
    app = ImageProcessor(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

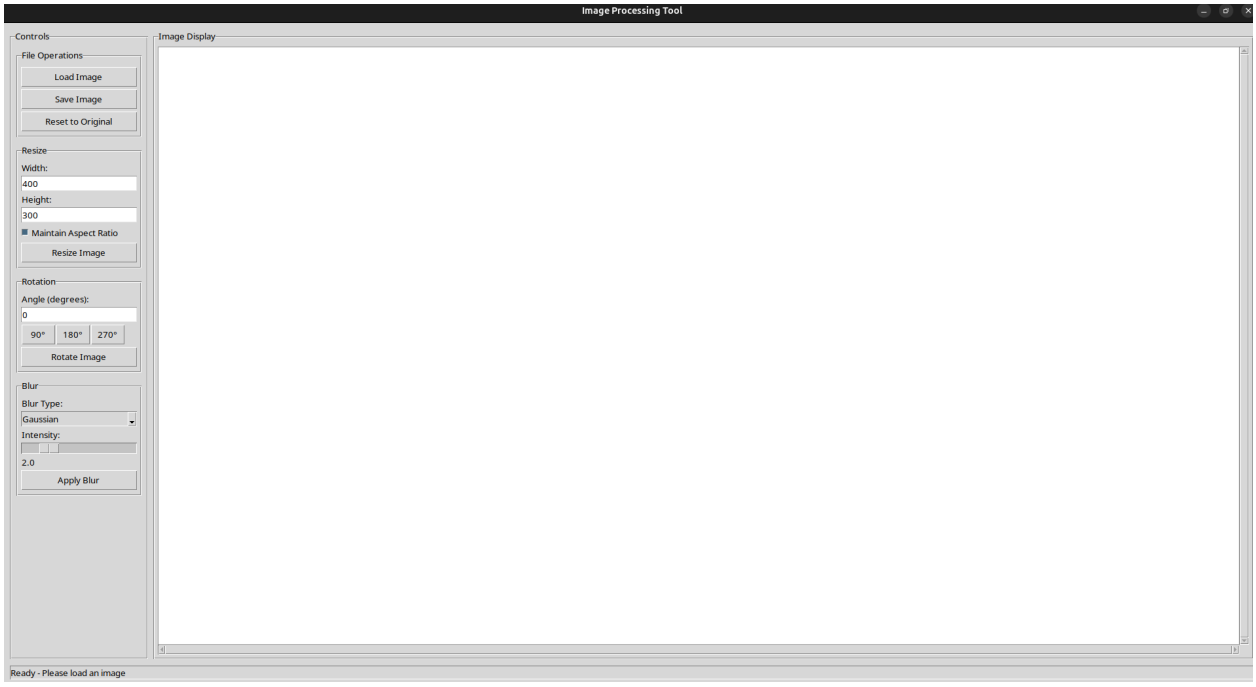
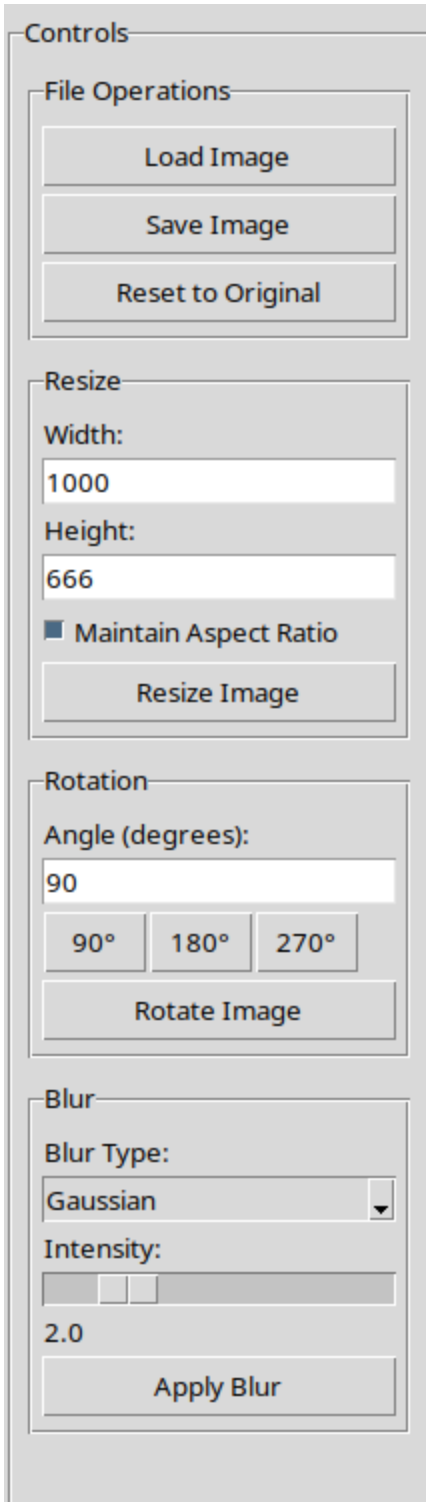
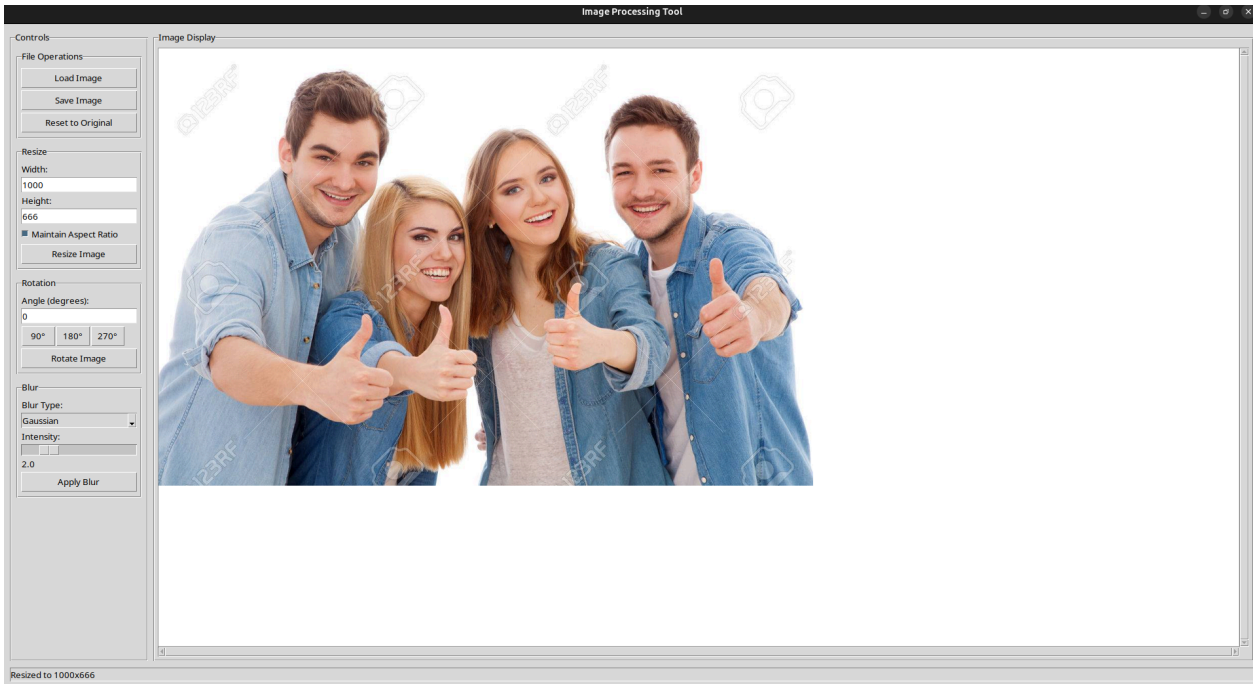


Image Processing Tool
Controls
File Operations
Load Image
Save Image
Reset to Original
Resize
Width:
400
Height:
300
Maintain Aspect Ratio
Resize Image
Rotation
Angle (degrees):
0
90°
180°
270°
Rotate Image
Blur
Blur Type:
Gaussian
Intensity:
2.0
Apply Blur
Image Display
Ready - Please load an image


Controls
File Operations
Load Image
Save Image
Reset to Original
Resize
Width:
1000
Height:
666
Maintain Aspect Ratio
Resize Image
Rotation
Angle (degrees):
90
90°
180°
270°
Rotate Image
Blur
Blur Type:
Gaussian
Intensity:
2.0
Apply Blur

Resize



Rotate

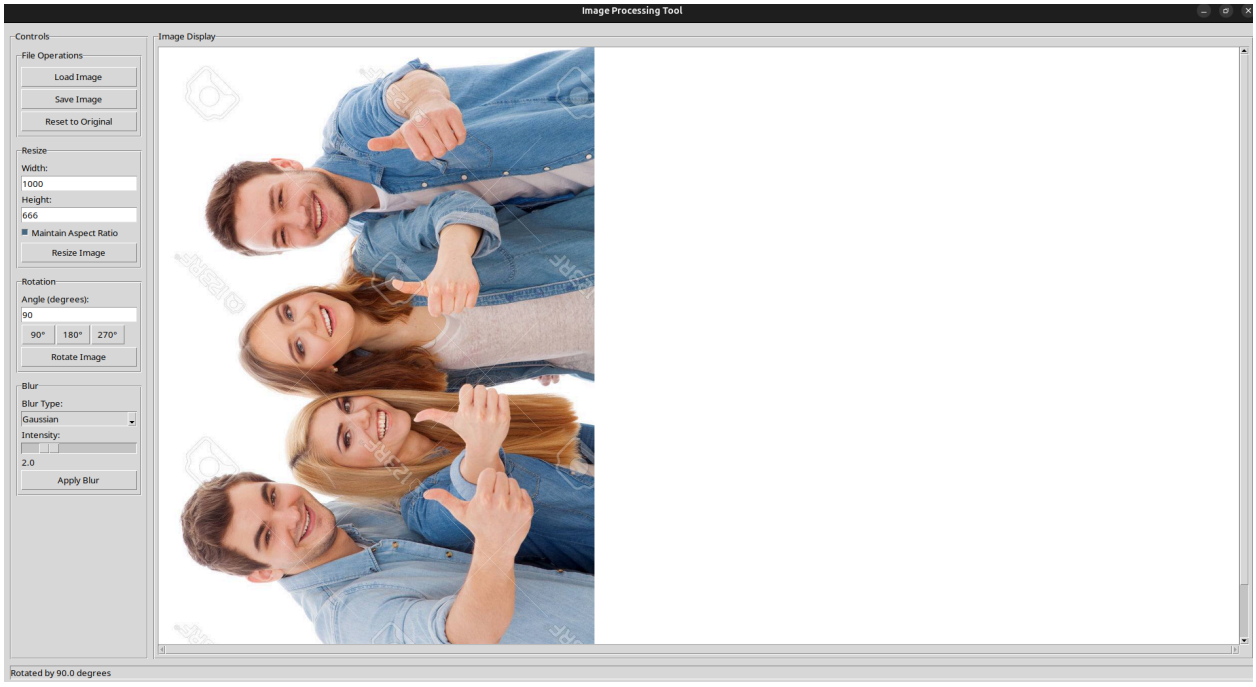


Image Processing Tool
Controls
File Operations
Load Image
Save Image
Reset to Original
Resize
Width:
1000
Height:
666
Maintain Aspect Ratio
Resize Image
Rotation
Angle (degrees):
90
90°
180°
270°
Rotate Image
Blur
Blur Type:
Gaussian
Intensity:
2.0
Apply Blur
Image Display
Rotated by 90.0 degrees

Blur


Image Processing Tool
Controls
File Operations
Load Image
Save Image
Reset to Original
Resize
Width:
1000
Height:
666
Maintain Aspect Ratio
Resize Image
Rotation
Angle (degrees):
90
90°
180°
270°
Rotate Image
Blur
Blur Type:
Gaussian
Intensity:
2.0
Apply Blur
Image Display
Applied Gaussian blur (intensity: 2.0)

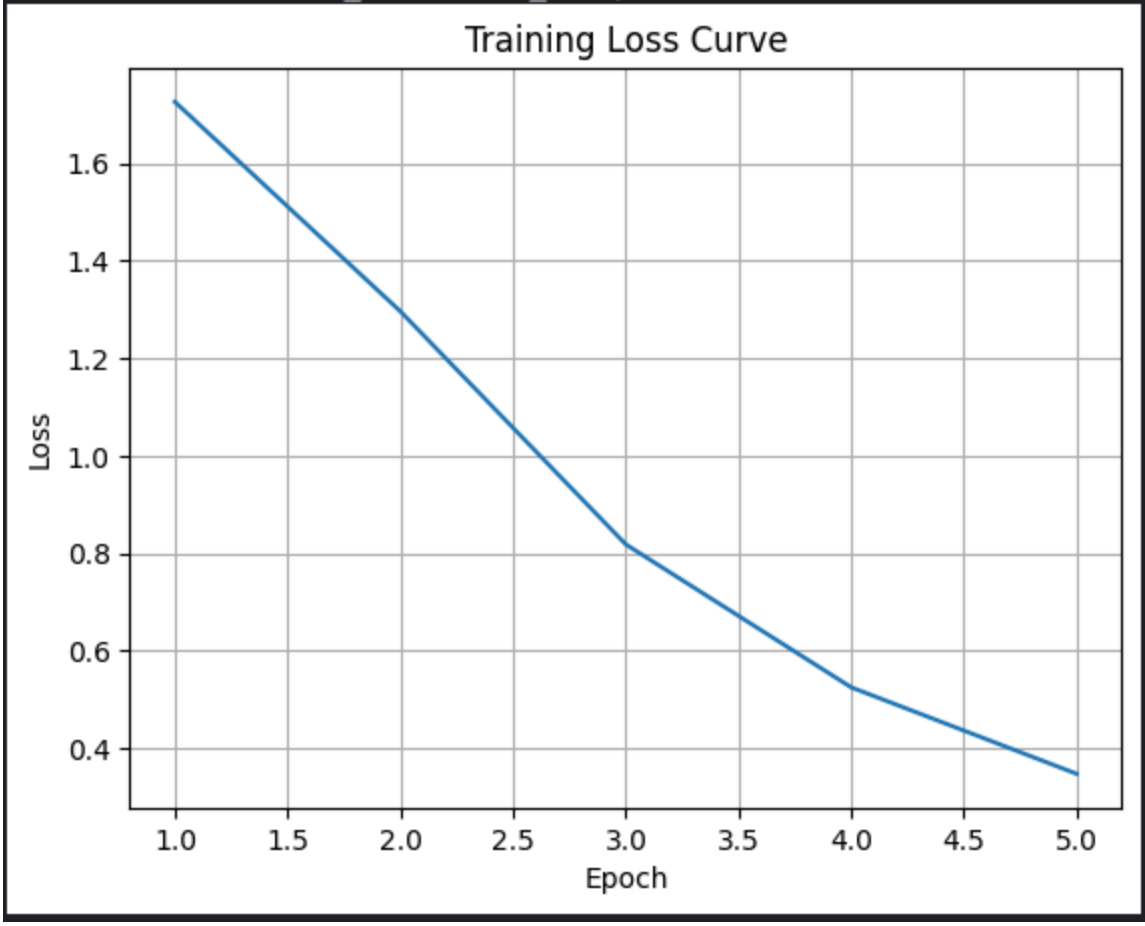
Q5. Create a basic face recognition system that can identify a person from a database of known faces

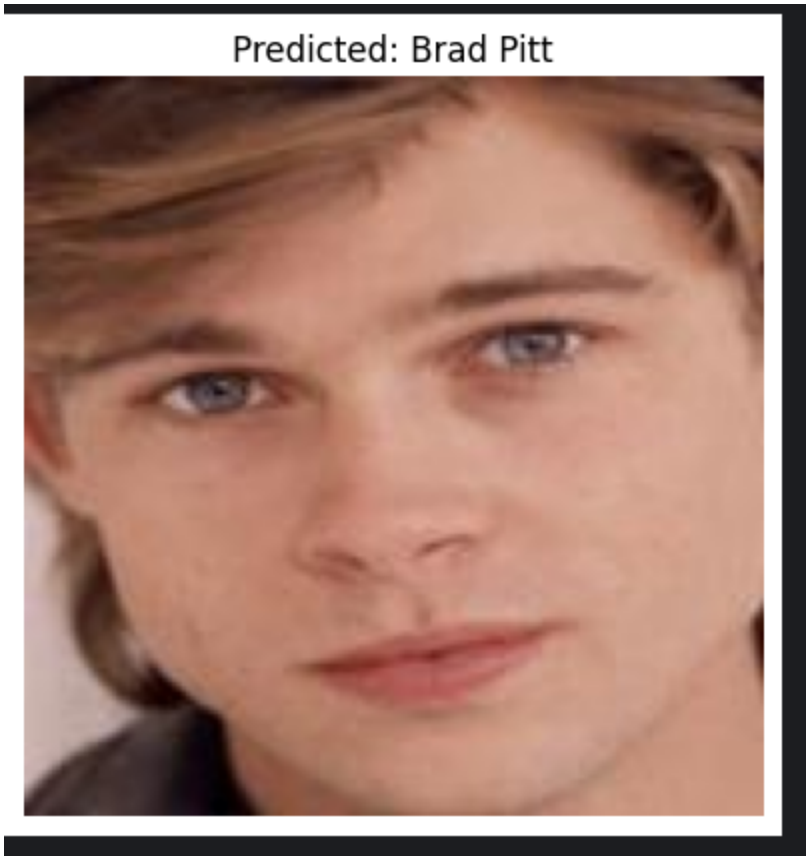
```
import os
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
from PIL import Image
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
data_dir = "/kaggle/input/facerecognitioncnn/Faces"
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5])
])

label_map = {
    0: "Amitabh Bachchan",
    1: "Brad Pitt",
    2: "Virat Kohli",
    3: "Robert Downey Jr",
    4: "Dwayne Johnson"
}

full_dataset = datasets.ImageFolder(root=data_dir, transform=transform)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_ds, val_ds = random_split(full_dataset, [train_size, val_size])
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=5):
        super(SimpleCNN, self).__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1),
```

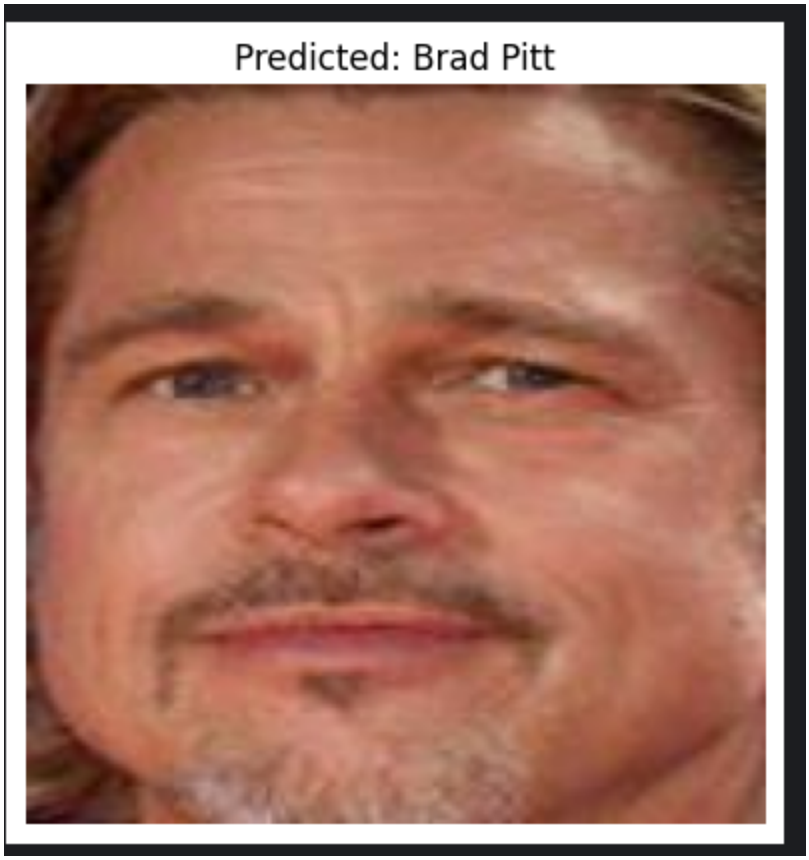
```
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Flatten(),
            nn.Linear(32 * 32 * 32, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )
    def forward(self, x):
        return self.net(x)

model = SimpleCNN(num_classes=5).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
epochs = 5
losses = []

for epoch in range(epochs):
    model.train()
    running_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    avg_loss = running_loss / len(train_loader)
    losses.append(avg_loss)
    print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")

torch.save(model.state_dict(), "face_classifier_cnn.pth")
print("Model saved as face_classifier_cnn.pth")

plt.plot(range(1, epochs+1), losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
```

```
plt.title("Training Loss Curve")
plt.grid(True)
plt.show()

class_dirs = os.listdir(data_dir)

for i in range(5):
   chosen_class = random.choice(class_dirs)
   image_file = random.choice(os.listdir(os.path.join(data_dir,
chosen_class)))
   image_path = os.path.join(data_dir, chosen_class, image_file)
   img = Image.open(image_path).convert("RGB")
   img_tensor = transform(img).unsqueeze(0).to(device)
   with torch.no_grad():
       output = model(img_tensor)
       _, pred = torch.max(output, 1)
       predicted_name = label_map[pred.item()]
   plt.imshow(img)
   plt.axis("off")
   plt.title(f"Predicted: {predicted_name}")
   plt.show()
```

```
Epoch 1/5, Loss: 1.7254
Epoch 2/5, Loss: 1.2961
Epoch 3/5, Loss: 0.8183
Epoch 4/5, Loss: 0.5253
Epoch 5/5, Loss: 0.3480
Model saved as face_classifier_cnn.pth
```




Predicted: Brad Pitt


Predicted: Brad Pitt

Predicted: Robert Downey Jr

Predicted: Virat Kohli

Predicted: Robert Downey Jr

Predicted: Dwayne Johnson